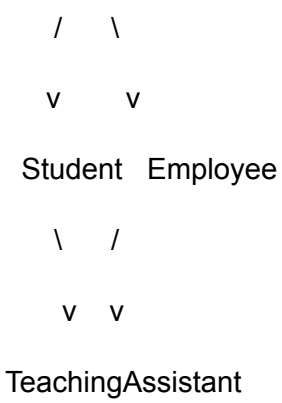
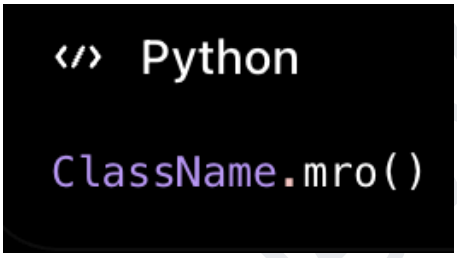
DAY-25 | PYTHON 90 DAYS CHALLENGE

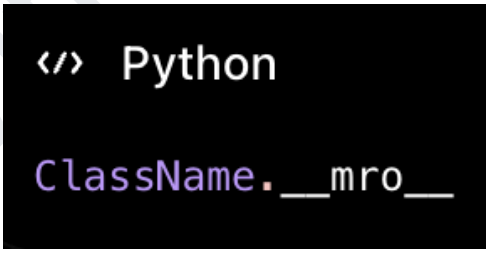


14.17 Method Resolution Order - MRO

MRO is the order Python follows while searching for methods in inheritance.

Syntax



OR


1. MRO decides which method is called first.
2. It is important in multiple inheritance.
3. Python uses the C3 linearization algorithm for MRO.
4. The search starts from the child class.
5. Then Python checks parent classes according to MRO.
6. Finally, it checks the base object class.

Example 1

PYTHON CODE
```
class A:
```

Code With Nishchal

```
    def show(self):
        print("A")

class B(A):
    def show(self):
        print("B")

class C(A):
    def show(self):
        print("C")

class D(B, C):
    pass

obj = D()
obj.show()
print(D.mro())
```

Output: B

[<class '__main__.D'>, <class '__main__.B'>, <class '__main__.C'>, <class '__main__.A'>, <class 'object'>]

MRO Flow

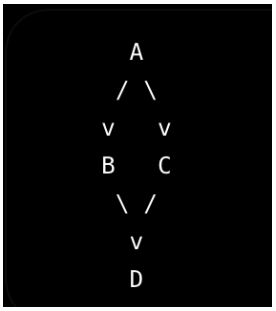
D -> B -> C -> A -> object

Since B comes before C, B.show() runs.

Diamond Problem

CodeWithNishchal

The diamond problem happens when a child class inherits from two classes that both inherit from the same parent.



1. Class B and class C both inherit from A.
2. Class D inherits from both B and C.
3. If the same method exists in multiple classes, Python uses MRO to decide.
4. Python handles this safely using MRO.
5. super() also follows MRO.

Example 1

PYTHON CODE

```
class A:
    def show(self):
        print("A")

class B(A):
    def show(self):
        print("B")

class C(A):
    def show(self):
        print("C")

class D(B, C):
    pass
d = D()
d.show()
```

Output: B
Because the MRO is:
D -> B -> C -> A -> object

Constructor in Inheritance

Nishchal

When a child object is created, Python runs the child class __init__ method.
If the child class does not have __init__, Python uses the parent class __init__.

Case 1: Child Has No __init__

PYTHON CODE

```
class Person:
    def __init__(self, name):
        self.name = name

class Student(Person):
    pass

student = Student("Aman")
print(student.name)
```

Output: Aman

The child class uses the parent class constructor.

Case 2: Child Has Its Own __init__

PYTHON CODE

```
class Person:
    def __init__(self, name):
        self.name = name

class Student(Person):
    def __init__(self, course):
        self.course = course

student = Student("Python")
print(student.course)
```

Output:

Python

Here, parent __init__ does not run automatically because the child has its own __init__.

Nishchal

super()

super() is used to call the next method in the MRO, usually from the parent class.

Syntax

super().method_name()

Explanation

1. super() is commonly used to call parent class methods.
2. It is commonly used inside __init__.
3. It avoids directly writing the parent class name.
4. In multiple inheritance, super() follows MRO.
5. It helps avoid repeating parent class logic.

Example 1

PYTHON CODE

```
class Person:
    def __init__(self, name):
        self.name = name

class Student(Person):
    def __init__(self, name, course):
        super().__init__(name)
        self.course = course

student = Student("Aman", "Python")

print(student.name)
print(student.course)
```

Output:

Aman

Python

Nishchal